

CUIオセロゲーム 内部設計書

1. 概要

本ドキュメントは、CUIオセロゲームの内部構造、クラス設計、およびモジュール間の連携について定義する。オブジェクト指向の原則に基づき、関心事の分離を目指す。

2. パッケージ構成

```
othello/
├── __init__.py
├── board.py      # 盤面の状態とロジック
├── game.py       # ゲーム全体の進行管理
├── disc.py       # 石の状態（黒、白、空）の定義
├── player/
│   ├── __init__.py
│   ├── player.py      # Playerの抽象基底クラス
│   ├── human_player.py # 人間プレイヤー
│   ├── computer_player.py # コンピュータプレイヤー（Context）
│   ├── strategy.py    # AI思考ロジックの抽象基底クラス（Strategy）
│   └── strategies.py  # 具体的なAI戦略の実装
└── main.py         # CUIアプリケーションのエントリーポイント
```

3. クラス設計

3.1. `othello.disc.Disc` (Enum)

- **役割:** 石の状態を表現する列挙型。
- **属性:** `EMPTY`, `BLACK`, `WHITE`

3.2. `othello.board.Board`

- **役割:** オセロの盤面状態の管理と、盤面に関するコアロジック（石の配置、裏返し処理など）責務を持つ。
- **主要な属性:**
 - `grid`: 8x8の2次元リスト。各要素は`Disc` Enum。
- **主要なメソッド:**
 - `__init__()`: 盤面を初期状態に設定する。
 - `place_disc(x, y, color)`: 指定した位置に石を配置し、相手の石を裏返す。
 - `is_valid_move(x, y, color)`: 指定した位置に石を配置できるか判定する。
 - `get_valid_moves(color)`: 指定した色の石が配置可能なすべての座標をリストで返す。
 - `_get_flippable_discs(x, y, color)`: (内部用) 指定位置に置いた場合に裏返る石のリストを取得する。

3.3. `othello.game.Game`

- **役割:** ゲーム全体の進行を管理する。`Board`インスタンスを所有し、現在のターンやゲームの終了判定を行う。
- **主要な属性:**
 - `board`: `Board`クラスのインスタンス。
 - `current_player`: 現在の手番の石の色 (`Disc Enum`)。
- **主要なメソッド:**
 - `__init__()`: `Board`を初期化し、最初の手番（黒）を設定する。
 - `play_turn(x, y)`: `board.place_disc`を呼び出し、成功したら`switch_player`を呼び出す。
 - `switch_player()`: 手番を交代する。
 - `is_game_over()`: ゲームの終了条件（盤面が埋まった or 両者パス）を判定する。
 - `get_winner()`: ゲーム終了時に勝者を判定する。

3.4. Player関連 (Strategyパターン)

3.4.1. `othello.player.player.Player` (ABC)

- **役割:** 全てのプレイヤー（人間、コンピュータ）の共通インターフェースを定義する抽象基底クラス。
- **主要なメソッド:**
 - `make_move(board)`: (抽象メソッド) 次の手を決定する。

3.4.2. `othello.player.human_player.HumanPlayer`

- **役割:** 人間プレイヤーを表現する。
- **メソッド:**
 - `make_move(board)`: 何も行わない（pass）。手はUI層からの入力で決定されるため。

3.4.3. `othello.player.strategy.ComputerPlayerStrategy` (ABC)

- **役割:** コンピュータの思考ロジックの共通インターフェースを定義する抽象基底クラス（Strategyインターフェース）。
- **主要なメソッド:**
 - `choose_move(board, color)`: (抽象メソッド) 次の手を選択する。

3.4.4. `othello.player.strategies.*`

- **役割:** `ComputerPlayerStrategy`を継承した、具体的な思考ロジック。
 - `RandomStrategy`: ランダムに手を選択する。
 - `GreedyStrategy`: 最も多くの石を裏返せる手を選択する。
- **メソッド:**
 - `choose_move(board, color)`: それぞれの戦略に基づいて手を実装する。

3.4.5. `othello.player.computer_player.ComputerPlayer`

- **役割:** コンピュータプレイヤーを表現し、具体的な戦略（`ComputerPlayerStrategy`）を保持する（Contextクラス）。
- **主要な属性:**
 - `_strategy`: `ComputerPlayerStrategy`のインスタンス。
- **主要なメソッド:**

- `make_move(board)`: 保持している `_strategy` オブジェクトに手の選択を委譲 (`_strategy.choose_move()` を呼び出す)。

3.5. `main.py`

- **役割**: CUIのUI層を担当するエントリーポイント。
- **責務**:
 - ユーザーからの入力受付と検証。
 - `Game` オブジェクトや `Player` オブジェクトの生成と管理。
 - ゲームループの実行。
 - `Game` オブジェクトの状態 (盤面など) をコンソールに表示する。

4. 処理フロー

1. **起動**: `main.py` が実行される。
2. **設定**: `main._select_game_settings()` が呼び出され、ユーザーの選択に応じて `player1` と `player2` に `HumanPlayer` または `ComputerPlayer` のインスタンスが設定される。`ComputerPlayer` には選択された `Strategy` が設定される。
3. **ゲーム開始**: `main.game_loop()` が開始される。`Game` オブジェクトが生成される。
4. **ターン処理**: a. `game_loop` は現在のプレイヤーが `HumanPlayer` か `ComputerPlayer` かを `isinstance` で判定する。 b. **Human**: ユーザーからの入力を待つ。入力された手で `game.play_turn()` を呼び出す。 c. **Computer**: `current_player.make_move()` (実体は `ComputerPlayer.make_move`) を呼び出して手を取得し、その手で `game.play_turn()` を呼び出す。 d. `game.play_turn()` は `board.place_disc()` を呼び出して盤面を更新し、成功すれば `game.switch_player()` を呼び出して `game.current_player` を更新する。
5. **ループ**: ゲーム終了条件 (`game.is_game_over()` が `True`) になるまで4を繰り返す。
6. **終了**: `main.show_result()` が呼び出され、結果が表示される。

この設計により、ゲームのコアロジック (`othello` パッケージ) とUI (`main.py`) が明確に分離され、保守性、拡張性が高められる。

Written by ポリテクセンター栃木